A Run-Time Reconfigurable Neural Inference Engine with Deterministic Sub-Microsecond Latency on a Low-Cost FPGA SoC

Ahmet Batuhan Özkaner[1*], Mustafa Alper Ilık[1] and Baykal Sarıoğlu[1]

[1*]Department of Electrical and Electronics Engineering, Istanbul Bilgi University, Istanbul, Türkiye.

*Corresponding author(s). E-mail(s): ahmet.ozkaner@bilgiedu.net;
Contributing authors: alper.ilik@bilgiedu.net; baykal.sarioglu@bilgi.edu.tr;

Abstract

Running neural inference on an embedded processor is flexible but leaves latency exposed to scheduling and the software stack; moving it into programmable logic makes latency predictable but normally ties the bitstream to one network, so every retrained model costs a synthesis run. This paper presents an inference engine that avoids both. The network shape is a run-time parameter held in configuration registers, so networks of different topologies are deployed into an unchanged bitstream over AXI-Lite with no synthesis step. On a low-cost Xilinx Zynq-7000 the engine uses 87.3 % of the DSP slices and 27.0 % of the LUTs at an estimated 2.70 W, closes timing at 125 MHz, and completes an inference in 768 ns. An on-chip cycle counter shows this latency to be invariant: over 100,000 inferences the standard deviation is zero. The same network run bare-metal on the SoC's Cortex-A9 requires 52.4 µs, 68 times longer. The datapath was verified bit-exactly against an independent reference model over 400,000 inferences across four topologies, a process that first exposed three pipeline-alignment defects invisible to synthesis. Against the same four networks generated by hls4ml for the same device, clock and arithmetic, over 20 design points, every one that fits placed and routed, no design of the two larger networks meets 125 MHz, and the fastest is 1.3 to 2.0 times slower than the engine even at its own maximum frequency. The engine is evaluated end to end over Ethernet on a latency-critical financial decision task.

Keywords: Embedded systems, FPGA, Run-time reconfiguration, Neural network inference, Deterministic latency

1 Introduction

Embedded systems increasingly evaluate a small neural network on every input event, under a latency bound that must hold for the worst case rather than on average. Running the network on the embedded processor keeps the model easy to change but exposes latency to scheduling, interrupts, caches and the software stack, so the distribution acquires a heavy tail. Moving the network into programmable logic removes those sources of variation, but the usual high-level-synthesis route binds the bitstream to one network: a retrained model, or a change of topology, costs a synthesis and place-and-route run measured in hours.

This paper presents an inference engine that keeps the determinism of the fabric without that rigidity. The engine runs in the programmable logic (PL) of a low-cost Zynq-7000 SoC, is fed over Ethernet through the processing system (PS) and an AXI-DMA / AXI-Stream datapath, and treats the network shape as a run-time parameter: layer widths live in configuration registers, so networks of different topologies are loaded into an unchanged bitstream over AXI-Lite with no synthesis step between them.

We evaluate it on a task that makes the latency requirement concrete. In high-frequency trading the time between market data and order matters as much as

the correctness of the order: Budish et al. [1] model the resulting latency race as an equilibrium feature of continuous limit order books, and Aquilina et al. [2] find races decided on the scale of microseconds. The workload is a three-class directional decision over Binance BTCUSDT trade data (Section 4.3); nothing in the architecture is specific to it.

1.1 Main Contributions

- A 192-multiplier, 13-stage pipelined MLP core for the XC7Z020 whose latency follows an analytic model ($\lceil n_\ell/3 \rceil + 14$ cycles per layer) that matches the oscilloscope and an on-chip cycle counter exactly in total and to within one cycle per layer.
- Run-time reconfigurability of the network *shape*, not only of its weights: four topologies run in one unchanged bitstream, each at its own cycle count.
- A three-level verification (independent bit-accurate model, directed RTL simulation, on-hardware comparison) that exposed three pipeline-alignment defects invisible to synthesis and then established bit-exact agreement over 400,000 inferences and 1,200,000 raw outputs.
- An end-to-end Ethernet-to-decision measurement with a per-stage breakdown over 10^6 packets, a zero-variance cycle count over 100,000 inferences, and like-for-like baselines on the same SoC.
- A measured comparison against hls4ml for the same four networks, part, clock and arithmetic, over five reuse factors taken through place and route.

2 Related Work

2.1 FPGA Acceleration in Financial Systems

Leber et al. [3] moved market-data feed handling and order generation into an FPGA and reported wire-to-wire latencies roughly an order of magnitude below a tuned software path; Lockwood and Monga [4] pursued the same argument at data-center scale. Puranik et al. [5] implemented an order-processing back end through high-level synthesis at roughly 500 ns per order, and Brown et al. [6] applied dataflow transformations to a credit-default-swap pricing engine. LightTrader [7] is closest in intent: an FPGA handles the network and order path while the learned model runs on separate deep-learning accelerator chips. In all of these the accelerated function is fixed at elaboration time, which suits stable algorithms but not a learned model that is retrained periodically. Learned order-book models [8, 9] are in turn evaluated offline and rarely report deployment latency.

2.2 Neural Network Inference on FPGAs

The FPGA deep-learning literature has converged on compiler-style frameworks that translate a trained network into a bitstream. Zhang et al. [10] formalised the underlying design-space exploration, Qiu et al. [11] and Angel-Eye [12] brought it to embedded Zynq parts, FINN [13] and FINN-R [14] generate dataflow architectures for quantized networks with latencies of a few hundred nanoseconds, and hls4ml [15] targets the same regime for particle-physics triggers, which share the hard latency budget of trading. hls4ml has since become a general compiler platform [16], and da4ml [17] replaces its constant matrix-vector products with distributed arithmetic. Both gain their efficiency from weights that are constants of the circuit. On the XC7Z020 used here, Vitis HLS has met a 70 µs worst-case budget for cognitive radio [18]; in this journal, Gdaim et al. [19] guide an OpenCL HLS flow on a Zynq-7000 with predictive latency and BRAM models, and Rangsikunpum et al. [20] run a binarized classifier on the same ZedBoard at over 9,000 frames per second within about 2.8 W. Edge CNN accelerators such as SparkNoC [21] optimize energy efficiency, Tsoutsouras et al. [22] apply the same methodology to a non-neural classifier, and Wang et al. [23] survey the field. On SoC targets the PS–PL data movement is often the limiting factor [24], which the breakdown of Section 5 reproduces. Fixed-point inference preserves accuracy for a wide range of networks [25–27], and quantization noise in MLPs can be predicted analytically [28].

All of these require a new synthesis run when the network changes, and all are evaluated as compute kernels with the network interface outside the reported latency.

2.3 Run-Time Programmable Overlays

Overlays remove re-synthesis by expressing the network as data for a fixed engine. VTA [29] defines a tensor instruction set and compiler, OPU [30, 31] compiles CNNs to microcode, and AMD's deep-learning processing unit [32] is a fixed overlay driven by compiled instructions; Khaki and Choi [33] use it for lightweight CNNs at 6.28 to 19.95 ms per frame. Kabir et al. [34] carry the idea to transformers, and Ratto et al. [35] use run-time reconfiguration to trade time for power within one network. SLAC's Neural Network

Library [36] keeps HLS-generated layers but reloads weights and biases at run time; its published description presents this as a weight reload rather than a change of layer dimensions. Dynamic partial reconfiguration rewrites the fabric itself: Boudjadar et al. [37] apply it on the XC7Z020 at 0.68 s per image, but every module must still be synthesized in advance and the region is out of service while it is rewritten.

We do not claim to have introduced run-time programmability. Published overlays are throughput architectures, dimensioned for convolutional workloads and reported per frame or per batch; among those that change the network shape without re-synthesis we are aware of none that reports single-inference latency below one microsecond. The present design accepts a narrower scope, fully connected layers within fixed bounds, for a latency that is a property of a single inference.

2.4 The Latency Extreme

LogicNets [38] compiles quantized neurons directly into lookup tables and reaches nanosecond latencies; later work learns piecewise polynomials per table [39], hides sub-networks inside Boolean functions [40], widens table inputs [41], assembles larger networks [42], and maps decision trees [43]. These designs are one to two orders of magnitude faster than the 768 ns reported here, but their weights are part of the circuit, so every retraining requires a new bitstream.

2.5 Positioning of This Work

The design space thus has three points: LUT-mapped networks give nanoseconds with per-model synthesis, overlays give re-programmability at per-frame throughput, and the present engine gives sub-microsecond single-inference latency with re-programmability on a low-cost part. The claim is the conjunction of four properties, none individually novel: (i) single-inference latency below one microsecond, given analytically and confirmed on hardware; (ii) different topologies in one unchanged bitstream; (iii) an Ethernet-to-decision path measured end to end and per stage; and (iv) bit-exact agreement with an independent reference over 4×10^5 inferences. Table 1 positions the work; its latency figures are not measured between the same points, and Section 8 supplies a like-for-like comparison with hls4ml.

3 Architecture and Implementation

3.1 System Overview

Fig. 1 shows the partition. Ethernet frames are received by the PS, which runs bare-metal lwIP and hands the feature vector to the PL through AXI-DMA; the `mlp_axi_wrapper` receives the stream, runs the core, and exposes the class scores over AXI-Lite. Each 32-bit stream word carries two 16-bit $Q8.8$ values, and the 40 features, zero-padded to the 64-wide datapath, arrive as 32 words on which the engine starts. Feature construction and quantization happen on the host, so the PS only copies the received vector into a DMA buffer; the argmax over the three scores runs on the PS after read-back and is counted in the readout stage of the latency breakdown. A model is deployed by quantizing it on the host and writing the weights, biases and layer widths over AXI-Lite; the bitstream is unchanged.

3.2 Inference Core

The core (`mlp_engine`, Fig. 2) uses 192 DSP slices as parallel multipliers; accumulation is done by a fabric adder tree rather than inside the DSP blocks, which is why logic utilization is dominated by the reduction network. The 13-stage pipeline has one address-issue stage (S0) followed by twelve datapath stages: S1–2 read and register weights and inputs, S3–4 multiply, S5–10 reduce through a balanced $64 \rightarrow 1$ tree, S11 adds the bias, and S12 rescales, saturates and applies ReLU. Each token carries three neurons of 64 inputs, and two ping-pong buffers hold the inter-layer activations so that every layer reuses the same multiplier array. Arithmetic is $Q8.8$ with 40-bit accumulation, an arithmetic right shift and 16-bit saturation; weights and biases reside in 195 block memories written through AXI-Lite (address bit 17 selects configuration or memory).

3.3 Measurement Infrastructure

The core exports a data-in pulse, one done pulse per layer and a busy signal on debug pins, so per-layer timing can be observed with an oscilloscope. A 16-bit cycle counter, cleared when `start` is accepted and latched when the final layer finishes, is read back over AXI-Lite after every inference. The firmware also timestamps each stage of the on-board path with the Zynq Global Timer and returns the timestamps in the reply.

Table 1 Comparison with representative FPGA-based financial-acceleration and neural-network inference studies. "Re-synth." indicates whether changing the deployed model requires a new bitstream. The reported metrics are not measured between the same two points and are given for orientation rather than as a ranking; see the discussion in the text.

Study	Platform	Function	Precision	Re-synth.	Net. I/O	Reported metric
Leber et al. [3]	Virtex-5	Feed handler, order gen.	n/a	Yes	Yes	2.6 μs half round trip
Puranik et al. [5]	Zynq	Order processing, HLS	n/a	Yes	No	~500 ns per order
Brown et al. [6]	Alveo U280	CDS pricing	Float	Yes	No	27,675 options/s
FINN [13]	ZC706	Binarized MLP/CNN	1 bit	Yes	No	0.31 μs, 12.3 M cls./s
hls4ml [15]	Virtex-7	MLP, trigger	Fixed	Yes	No	~100 ns, jet tagging
Angel-Eye [12]	Zynq XC7Z045	CNN compiler	8/16 bit	Yes	No	137 GOP/s, VGG16
SparkNoC [21]	Arria 10 GX1150	Lightweight CNN	Fixed	Yes	No	337.2 GOP/s
VTA [29]	Zynq, Ultra96	Tensor overlay	8 bit	No	No	Per-frame throughput
OPU [30]	Zynq XC7Z100	CNN overlay	8 bit	No	No	Per-frame throughput
LogicNets [38]	Virtex UltraScale+	LUT-mapped MLP	2 to 6 bit	Yes	No	~10 ns
This work	Zynq XC7Z020	MLP, run-time shape	16 bit	No	Yes	768 ns core, 68.6 μs end to end

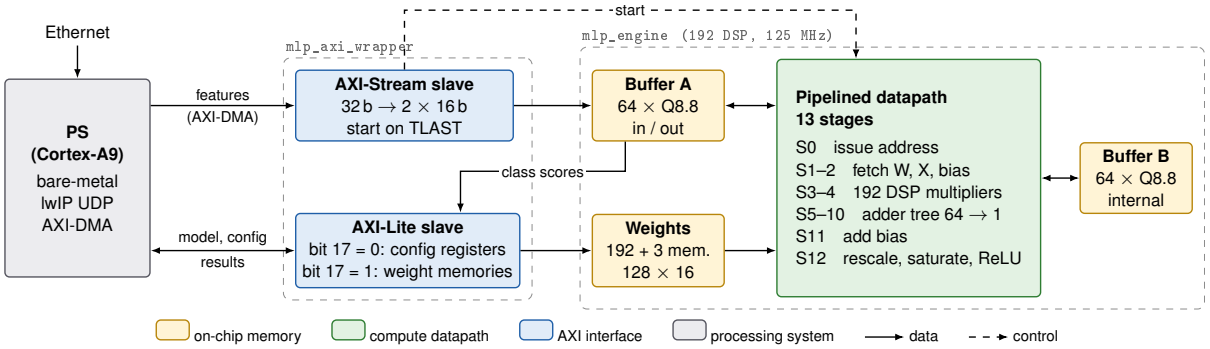


Fig. 1 System architecture of the proposed framework on the Zynq-7000 SoC. Market data arrives over Ethernet at the PS, which forwards the prepared feature vector to the custom `mlp_axi_wrapper` core through AXI-DMA / AXI-Stream; the model parameters are loaded and the final decision is read back over AXI-Lite. The entire inference is executed by the 13-stage pipelined inference engine in the PL.

4 Experimental Evaluation

4.1 Experimental Setup

Table 2 summarizes the setup. A host application (Fig. 3) uploads the quantized model and streams market data or test packets to the board. Two kinds of run are reported. The host round-trip runs time each UDP request at the host, over 100,000 requests for each of the four networks of Section 4.2, from a native Linux host and from a Windows host; they underlie Table 6 and Section 5.2. A longer instrumented run of 1,004,229 packets underlies the per-stage breakdown of Table 8. The two give medians of 66.9 and 68.5 µs for the reference network because they were taken at different times; their on-board subtotals agree to within 0.4 µs, and no claim depends on the difference.

4.2 Functional Verification of the Inference Core

4.2.1 Golden Reference Model

A bit-accurate software model was written independently of the RTL. It reproduces the host $Q8.8$ conversion with saturation, the truncating weight conversion of the loader, the 40-bit accumulation, the shift, the 16-bit saturation, ReLU on all but the output layer, and the argmax rule. A second, cycle-accurate model of `mlp_engine` reproduces the issue FSM, the token pipeline, the one-cycle memory latency and the ping-pong write-back.

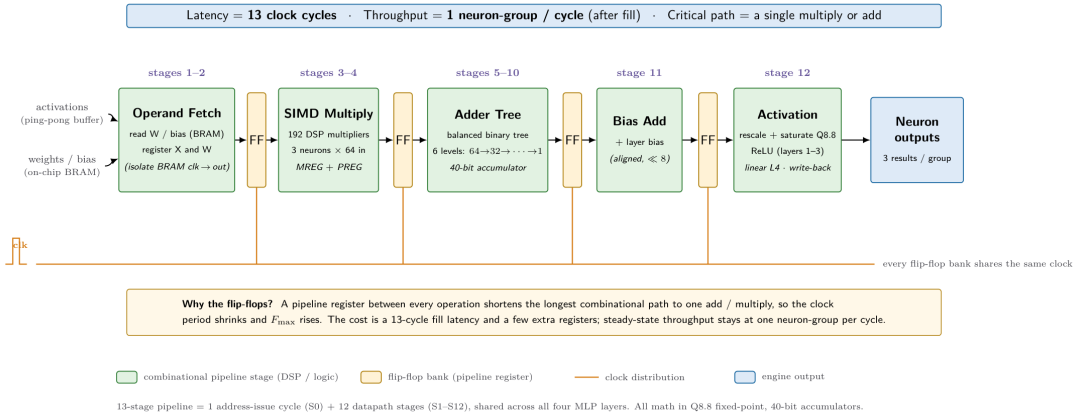


Fig. 2 Pipeline datapath of the hardware MLP core (`mlp_engine`). After the address-issue stage (S0), stages 1 and 2 fetch the weights, inputs, and bias from BRAM; stages 3 and 4 perform the 192 parallel $Q8.8$ multiplications on the DSP slices; stages 5 to 10 form the balanced $64 \rightarrow 1$ adder tree; stage 11 adds the bias; and stage 12 rescales and applies the ReLU before the result is written back to the ping-pong activation buffers. The `mlp_axi_wrapper` splits each 32-bit AXI-Stream word into two 16-bit $Q8.8$ values and triggers the engine; weights/biases reside in BRAM and are loaded via AXI-Lite (address bit 17 selects configuration vs. memory).

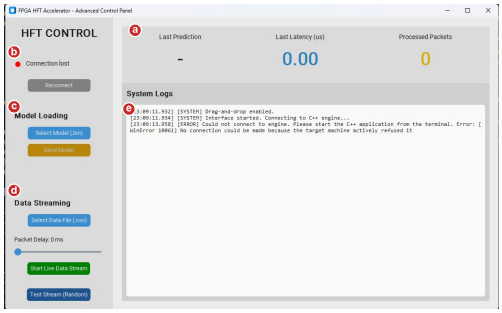


Fig. 3 Host control application: (a) live decision, round-trip latency and packet count, (b) engine connection state, (c) model upload, (d) stream source and inter-packet delay, (e) event log. Deploying a different network is the single action in panel (c); no synthesis step is involved.

4.2.2 Directed RTL Simulation

Three directed tests isolate the multiply-reduce-rescale path, the bias pipeline and the write-back indexing. Each writes its parameters through the same AXI-Lite map as the loader, streams a stimulus over AXI-Stream, and compares the outputs against hand-computed values; unused input positions are preloaded with distinctive values so that a missing write cannot pass as correct. The tests exposed three pipeline-alignment defects (Table 3), each a one-line index correction that changes neither depth, resources nor timing. After correction the cycle-accurate model matched the golden reference on 240 random cases (60 parameter sets $\times$ 4 inputs); the original alignment matched none.

Table 2 Experimental setup

Item	Description
FPGA device	Xilinx XC7Z020-1CLG484 (ZedBoard)
Design language	VHDL (PL); C++/Python (PS/host)
Tool version	Xilinx Vivado / Vitis 2025.2
PS software	Bare-metal, lwIP raw API (no OS on the board)
Data path	Ethernet $\rightarrow$ PS $\rightarrow$ AXI-DMA $\rightarrow$ AXI-Stream $\rightarrow$ PL
Inference core	MLP 40-64-32-16-3, $Q8.8$, 192 DSP multipliers
Dataset	Binance BTCUSDT aggregated trades, 16,631,211 records, 2025-05-01 to 2025-05-18
Task	3-class direction, horizon 50 trades, $\theta = 5 \times 10^{-5}$
Validation split	Final 10 % chronologically, 1,663,113 samples
Host round-trip runs	$4 \times 100{,}000$ requests per host (native Linux, Windows)
Instrumented run	1,004,229 packets (PS-timestamped, per stage)
Clock frequency	125 MHz (PL)

4.2.3 On-Hardware Bit-Exact Verification

The corrected design was verified on the board against the golden model with a stimulus built to exercise saturation, ReLU, sign transitions and zero padding. Recorded market data cannot do this: its features are clipped to $[-5, 5]$ and never drive the accumulator toward saturation. The firmware returns the three raw

Table 3 Pipeline-alignment defects found by directed simulation

Signal	Stage	Consequence
Bias pipeline	$11 \rightarrow 9$	Each neuron received the bias of a neuron six positions earlier
Write-back token	$12 \rightarrow 11$	Results were written to the wrong activation indices
Input multiplexer	$2 \rightarrow 1$	Each layer read from the wrong ping-pong buffer

16-bit outputs with every decision, so the comparison is made on the arithmetic and not only on the argmax. Across four networks the hardware reproduced the golden model on all 400,000 inferences and 1,200,000 raw outputs (Table 4).

A reconfigurable engine needs one further check: if loader and model both padded every network to the largest shape, they would agree while the shape mechanism went untested. The golden model therefore evaluates each network at its true topology, the loader writes the per-layer width registers before each run, and the cycle counter records how many iterations the controller issued. The four counts in Table 4 are distinct and each one cycle below the prediction of (1), the offset being the counting convention of Section 5.5; a padded network would have produced four equal counts. The runs used the bitstream characterized in Section 7.1, compared byte for byte against the boot image.

Table 4 On-hardware bit-exact verification against the golden model. All four networks were loaded at run time into the *same* bitstream, without a reset between them. "Outputs" compares the three raw 16-bit outputs of every inference, i.e. 1,200,000 individual comparisons in total, none of which differed. "Cycles" is the on-chip counter value, identical on every inference of a run and equal to $\sum_\ell \lceil n_\ell/3 \rceil + 55$.

		Mismatches		
Network	Inferences	Decision	Outputs	Cycles
40-64-32-16-3	100,000	0	0	95
40-64-64-32-3	100,000	0	0	111
40-32-16-8-3	100,000	0	0	76
40-10-16-8-3	100,000	0	0	69
Total	400,000	0	0	

4.3 Dataset and Classification Task

The workload is a three-way directional classification on the public Binance BTCUSDT aggregated-trade archive: 16,631,211 trades between 2025-05-01 and 2025-05-18. The 40 features come from a trailing window of 40 trades and are dimensionless: 20 slots of relative price changes scaled by 1000 (one of which is a constant zero left by zero-padding of the difference series), 10 quantities divided by the window mean, and 10 taker-side indicators of ±1. Every feature is clipped to $[-5, 5]$, so the integer part of $Q8.8$ has more than four bits of headroom. With $r_t = (P_{t+50} - P_t)/P_t$, the label is *buy* for $r_t > \theta$, *sell* for $r_t < -\theta$ and *hold* otherwise, with $\theta = 5 \times 10^{-5}$; larger thresholds leave the minority classes at 5 % or below 0.1 %. The split is chronological: the final 10 %, 1,663,113 samples, is held out (no embargo was applied; 50 boundary labels overlap the validation period). The validation period contains 57.16 % hold, 21.50 % buy and 21.35 % sell, so the majority-class baseline is 57.16 %.

4.4 Model Accuracy

Table 5 compares floating-point and $Q8.8$ accuracy for five networks. The fixed-point column comes from the bit-accurate model, which Section 4.2 shows to agree with the hardware bit for bit. Floating point and $Q8.8$ agree to within 0.22 percentage points, and to within 0.1 for the reference network, so the 16-bit datapath does not change the decisions materially. The reference network exceeds the majority baseline by 4.58 points; with a standard error of 0.04 points the three larger networks are well above sampling noise, while the two smallest are not distinguishable from the baseline (D) or fall below it (E). Recall of the reference network is balanced across classes (0.59 to 0.64) and precision is lower on the minority classes (0.51 to 0.57 against 0.70 for hold). Predictive quality is not a contribution of this work and no profitability is claimed; the task fixes a realistic network size and input rate.

5 Latency and Determinism

5.1 System-Level Packet-to-Decision Latency

Table 6 reports the UDP round trip from a native Linux host for all four networks in one bitstream and one session. For the reference network the median is 66.9 µs, of which the 768 ns core is about 1 %.

The columns differ on the board only where the architecture says they should: the median on-board

Table 5 Floating-point against $Q8.8$ accuracy on the 1,663,113-sample validation period, and the margin over the 57.16 % majority-class baseline. Model A is the reference network used throughout the paper.

Model	Topology	Float (%)	$Q8.8$ (%)	$\lvert\Delta\rvert$ (pp)	vs. base (pp)
A	40-64-32-16-3	61.74	61.65	0.09	+4.58
C	40-64-48-32-3	60.62	60.54	0.08	+3.46
B	40-64-64-32-3	60.58	60.59	0.02	+3.42
D	40-32-16-8-3	57.10	56.88	0.22	−0.06
E	40-10-16-8-3	56.86	57.01	0.15	−0.30

subtotal rises from 9.48 to 9.98 µs, entirely in the PL stage and in step with the cycle counts, which the firmware reads at the 250 ns granularity of its AXI-Lite polling. Reception, parsing, DMA and readout agree to within 0.01 µs. The three larger networks have medians within 0.7 µs of one another; the smallest, run first in the session, is 5 µs lower, and the cause is not on the board. In that run 38 % of requests finish below 60 µs, against 3 % for the others, and in that fast group the on-board time is the same 9.48 µs as in the slow group; the difference (44.7 against 51.8 µs off board) lies in the host, its NIC and the wire. The spread between columns is therefore a measure of how much the host alone moves the end-to-end figure between runs.

Table 6 Measured end-to-end packet-to-decision latency of the FPGA system from a native Linux host, 100,000 requests per network, all four networks in one unchanged bitstream (all values in μs). The board row is the median of the per-packet on-board subtotal (PS + PL); the cycle row is the core cycle count read back with every inference.

Hidden layers	10-16-8	32-16-8	64-32-16	64-64-32
Minimum	52.4	52.5	53.2	53.6
Median	61.5	66.4	66.9	67.1
Mean	62.7	67.6	68.2	68.2
99th percentile	81.3	79.8	80.9	81.8
99.9th percentile	146.1	116.2	113.6	112.8
Maximum	3260.4	3265.0	3260.9	3244.4
Board median	9.48	9.50	9.75	9.98
Core cycles	69	76	95	111

5.2 How Much of the End-to-End Latency Belongs to the Host

The same runs were repeated from a Windows host with board, bitstream and networks unchanged (Fig. 4). For the reference network Linux completes a median request in 66.9 µs (p99 80.9 µs, p99.9 113.6 µs); Windows needs 460.0 µs (p99 656.0 µs, p99.9 1878 µs), and 255 of its 100,000 requests exceed 1 ms, against 30 on Linux. The Windows medians of the four networks lie between 456.1 and 461.0 µs, so the network shape is invisible there. A factor of about seven in the median, and the whole heavy tail, originate on the host side. This is not a software baseline for the inference, which runs in the fabric in both cases; that comparison is in Section 6.2.

5.3 Core Latency and Cycle Model

The per-layer done pulses give the inter-pulse times of Fig. 5 and the total interval of Fig. 6: 280, 200, 160 and 118 ns for layers 1 to 4, and 768 ns in total. These values follow from the architecture. The issue FSM emits one token of three neurons per cycle, so a layer of n_ℓ outputs needs $\lceil n_\ell/3 \rceil$ issue cycles, and then waits for the pipeline to empty:

$$c_\ell = \left\lceil \frac{n_\ell}{3} \right\rceil + D + 1, \qquad D = 13. \tag{1}$$

Analytic models of this kind steer design-space exploration for systolic accelerators to within about one percent [44]; for a single-issue pipeline without data-dependent control the model can be exact. The predicted total, 96 cycles or 768 ns, equals the measured interval (Table 7). Two per-layer readings are below the prediction by less than one clock period, consistent with the reading uncertainty of separate captures; the total is corroborated independently by the cycle counter (Section 5.5).

Of the 96 cycles, 56 drain the pipeline between layers, so only 42 % of the core latency issues work; overlapping drain and issue would need sub-layer double buffering and is left as future work. With 192 multipliers at 125 MHz the peak is 48 GOP/s; the reference network performs 5168 useful multiply-accumulates while the zero-padded array issues 7680, so the achieved rate is $2 \times 5168/768\,\text{ns} \approx 13.5$ GOP/s, 28 % of peak. The core sustains 1.30 M inferences per second, and the system about 0.77 M because the wrapper does not overlap reception with computation; both far exceed what the host path supplies.

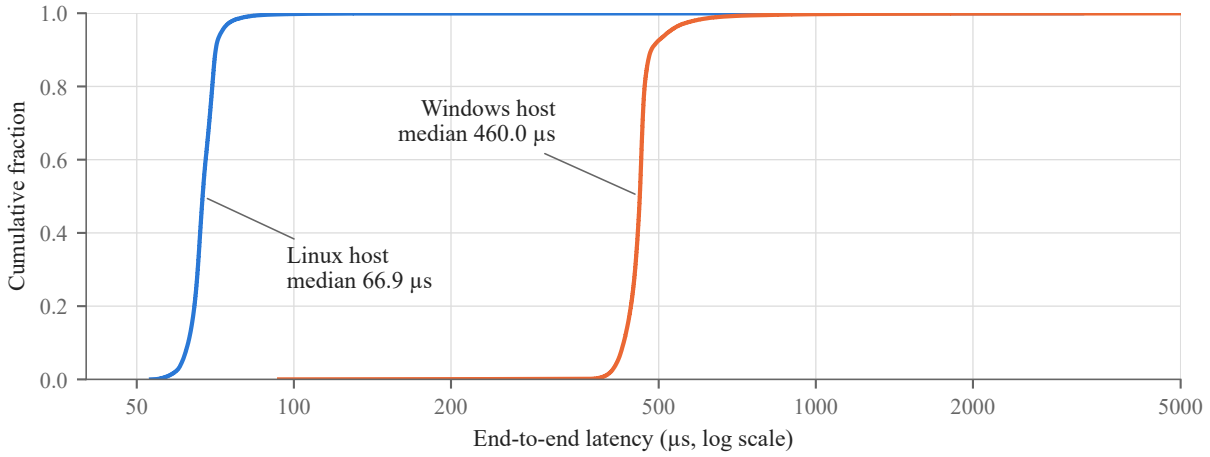


Fig. 4 Cumulative distribution of the end-to-end latency for the reference network, 100,000 requests per host, same board, bitstream and model.

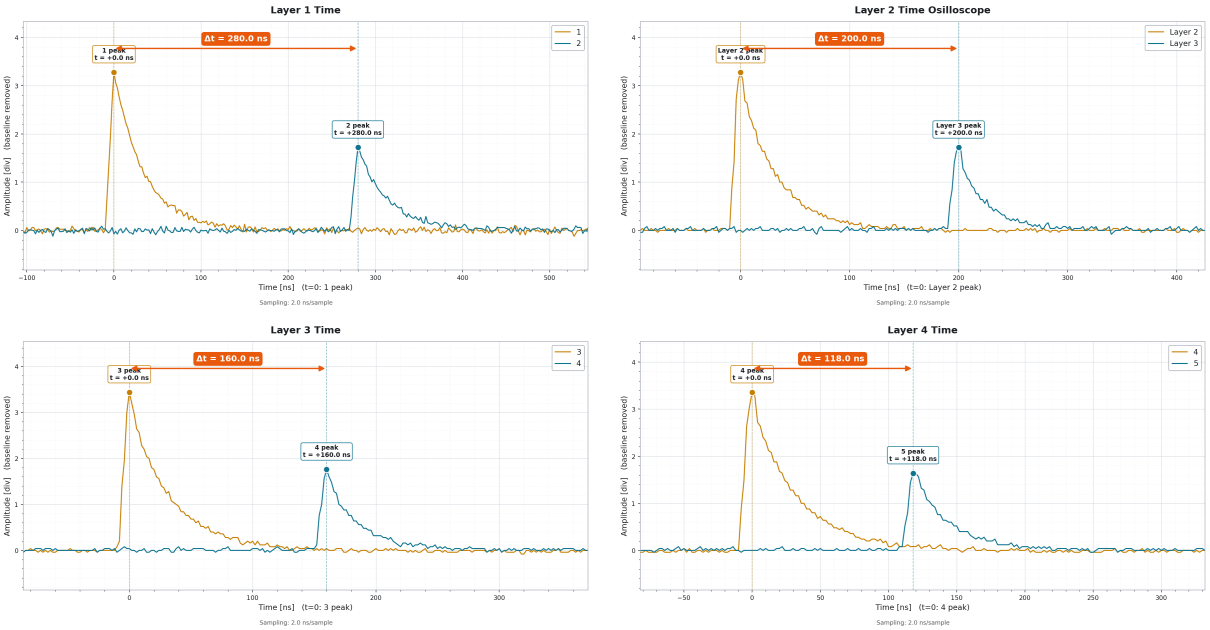


Fig. 5 Oscilloscope captures of consecutive layer-done debug pulses; each inter-pulse interval directly measures one layer's processing time. Top-left to bottom-right: layer 1 (280 ns), layer 2 (200 ns), layer 3 (160 ns), layer 4 (118 ns).

5.4 Per-Stage Breakdown of the End-to-End Path

Table 8 decomposes the path over 1,004,229 packets. Of the 68.6 µs mean, 14.4 µs is spent on the board and the rest in the host NIC, operating system and wire. On the board, Ethernet reception dominates and is bimodal (p50 2.6 µs, p99 14.2 µs), reflecting the phase of the polling loop, while DMA, PL and readout are tightly bounded. The PL stage is the interval seen by the processor, resolved in 250 ns polling steps and including the AXI-Stream transfer; the core latency itself is given by the counter and the debug pins.

5.5 Determinism of the Hardware Core

Over 100,000 consecutive inferences at line rate the cycle counter returned 95 every time: minimum, maximum and mean are equal and the standard deviation is zero, irrespective of the input values, the activation pattern and concurrent AXI-Lite and DMA traffic. The counter reads 95 where (1) predicts 96 because it

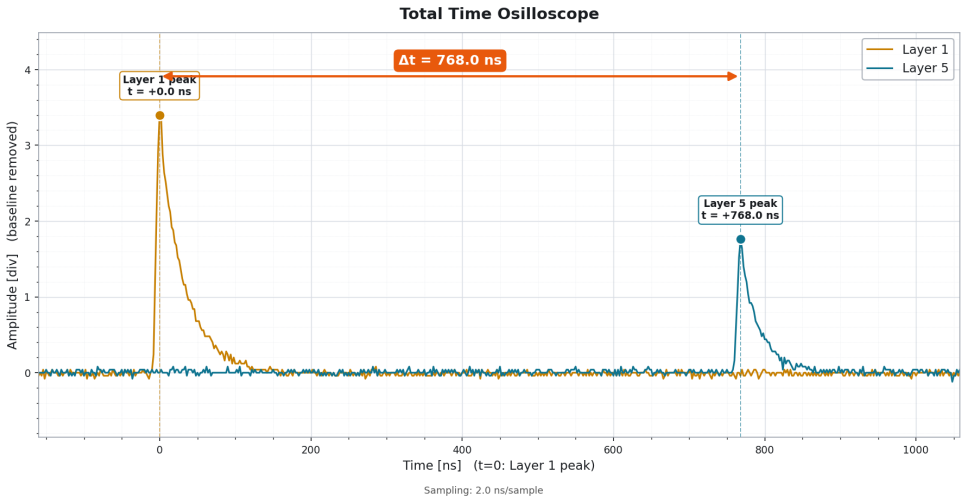


Fig. 6 Oscilloscope capture of the first and last debug pulses of an inference, measuring the total hardware-core interval ($\Delta t = 768$ ns).

Table 7 Per-layer latency of the MLP core at 125 MHz: analytic prediction from (1) against oscilloscope measurement on the layer-done debug pins. Layers 1 to 4 are $40 \to 64$, $64 \to 32$, $32 \to 16$ and $16 \to 3$ respectively.

			Predicted		Meas.
Layer	n_ℓ	$\lceil n_\ell/3 \rceil$	cyc	ns	ns
1	64	22	36	288	280
2	32	11	25	200	200
3	16	6	20	160	160
4	3	1	15	120	118
Total			96	768	768

counts only non-idle cycles and so excludes the cycle in which `start` is sampled; the start-to-done convention is used throughout.

6 Baselines on the Same Device

6.1 Sequential Hardware Baseline

An earlier version of this design, preserved in the artifact history, used a sequential FSM driving a single multiply-accumulate unit with the network dimensions hardwired. Each neuron costs one cycle for the bias, one per input and three to activate, write and advance, so a layer occupies $n_{\text{out}}(n_{\text{in}}+4)$ cycles; with layer transitions and a hardware argmax the design required 5639 cycles, 56.4 µs at its 100 MHz clock. At a common 125 MHz it would need 45.1 µs, so the architecture alone accounts for a factor of 58.7; as deployed the factor is 73.4, the remainder being the clock. The factor compares controller cycle counts, 5639 against 96, and comes from replacing the single unit with 192 parallel multipliers. The earlier design also received its inputs through one AXI-Lite write per operand, which the DMA-fed stream replaces; that change is outside both cycle counts and does not contribute to the factor.

6.2 Software Inference on the Cortex-A9

The same network was implemented in C and run bare-metal on the 667 MHz Cortex-A9 with identical $Q8.8$ arithmetic, timed with the Global Timer over 1000 inferences with warm caches and no network or DMA activity. It requires 52.4 µs on average, 68 times the fabric. The figure is for scalar `-O3` code on one core; without optimization the same source needs 302 µs. It corresponds to about 34,900 cycles for 6704 multiply-accumulates (the first layer is padded to 64 inputs), 5.2 cycles each; a NEON implementation would narrow the gap and its absence is a limitation.

The distributions differ more than the means. Software ranges from 51.2 to 67.1 µs (median 52.0, p99 52.6 µs), a spread more than twenty times the whole hardware inference, under conditions more favorable than any deployment; trained weights change the mean by about two percent because ReLU is a data-dependent branch. The hardware has no spread at all. The sequential hardware baseline, at 56.4 µs, is slightly slower than optimized software, so the gain comes from the parallel issue structure rather than from the change of substrate. The software path also occupies the processor for its whole duration, whereas the hardware path frees it as soon as the DMA descriptor is posted.

Table 8 Latency breakdown from Ethernet packet to trading decision, over a 1,004,229-packet run (all values in μs). Percentiles on the subtotal row are computed over the per-packet sums, not by adding the per-stage percentiles; the two differ because a packet slow in one stage is not in general slow in the others (adding the per-stage p99 values would give 21.51 rather than 21.03).

Stage	Mean	p50 / p99	Measurement Method
Ethernet packet reception (EMAC + lwIP)	7.59	2.57 / 14.18	PS timestamping (bimodal)
Packet parsing and buffer hand-off	0.70	0.69 / 0.86	PS timestamping
AXI-DMA transfer to PL	3.70	3.70 / 3.78	PS/PL counters
AXI-Stream receive + MLP core (PL)	1.46	1.46 / 1.72	dbg pins (core 768 ns)
Result readout	0.96	0.96 / 0.97	AXI-Lite
Board subtotal (PS + PL)	14.40	9.41 / 21.03	On-board sum
Host end-to-end	68.56	68.50 / 80.92	Host round trip

7 Implementation Results

7.1 FPGA Resource Utilization

Table 9 reports post-implementation usage from the routed checkpoint whose bitstream was verified in Section 4.2. The design is DSP-bound (87.3 %); LUTs are at 27.0 %, flip-flops at 10.7 % and block-RAM tiles at 70.4 %. The engine accounts for 13,038 LUTs, 9316 flip-flops, all 192 DSP slices and 195 RAMB18 primitives; the rest is DMA, interconnect and wrapper. The block-RAM figure counts tiles, not data: each of the 192 weight memories holds only 128 words of 16 bits in an 18-kbit primitive, so about 88 % of the allocated capacity is unused.

Table 9 Post-implementation resource utilization on the Xilinx XC7Z020-1CLG484

Resource	Used	Available	Utilization
Slice LUTs	14,366	53,200	27.00%
as logic	14,282	53,200	26.85%
as memory	84	17,400	0.48%
Slice registers	11,396	106,400	10.71%
Block RAM tiles	98.5	140	70.36%
DSP slices	192	220	87.27%
Bonded IOB	14	200	7.00%
BUFGCTRL	1	32	3.13%

7.1.1 Timing

All constraints are met at 125 MHz (Table 10): setup WNS 0.056 ns, hold WHS 0.027 ns, no failing endpoints, for an $f_{\max}$ of 125.9 MHz. The critical path is not arithmetic: it runs from an AXI SmartConnect register to the input of one of the 192 weight memories, 93.1 % of its 7.532 ns being routing. The worst path inside the engine (slack 0.113 ns) is likewise a fan-out from a write-back register to the activation buffer. Consolidating inputs into wider memory words would shorten both; it was not pursued because 125 MHz is supplied directly by the PS and the core is not the dominant latency term.

Table 10 Post-implementation timing closure at 125 MHz (8.000 ns period, speed grade −1)

Check	Worst slack	Failing endpoints
Setup (WNS)	0.056 ns	0 / 31,890
Hold (WHS)	0.027 ns	0 / 31,890
Pulse width (WPWS)	3.020 ns	0 / 12,115

7.1.2 Power

Vivado's vectorless estimate is 2.704 W on chip (Table 11), a tool estimate rather than a measurement. The PS accounts for 1.535 W; the whole programmable-logic contribution is 0.974 W, which is the figure to compare against accelerator cards that include no application processor. The ZedBoard's XC7Z020 costs in the low hundreds of US dollars, so a deterministic sub-microsecond path for a network of this size does not require a data-center part.

7.2 Run-Time Reconfigurability and Scalability

The per-layer output widths are AXI-Lite registers that the issue FSM reads to derive its iteration counts, so the network shape is a run-time parameter. The four networks of Table 4, with first hidden layers from

Table 11 On-chip power, Vivado vectorless estimate for the implemented design

Component	Power
PS7 (processing system)	1.535 W
Signals	0.272 W
Block RAM	0.249 W
DSP	0.236 W
Slice logic	0.145 W
Clocks	0.067 W
I/O	0.005 W
Dynamic total	2.509 W
Device static	0.196 W
Total on-chip	2.704 W
of which PL only	0.974 W

10 to 64 neurons, were loaded into the same programmed device and each verified bit-exactly, without re-synthesis or bitstream reload. Loading a network takes $\sum_\ell \lceil n_\ell/3 \rceil \times 3 \times 64$ weight writes plus biases, about 7800 AXI-Lite transactions for the reference topology.

Because all 64 inputs of a neuron slot are multiplied in parallel, only the number of output neurons enters the cycle count, so the four networks complete in 69, 76, 95 and 111 counted cycles, 560 ns to 896 ns start to done, each constant over its run. The drain term of 4×14 cycles does not depend on the shape, so it exceeds half the latency of the smaller networks. The datapath is elaborated for at most four layers of at most 64 neurons and 64 inputs; any network within those bounds is hosted without re-synthesis. Beyond them the limit is memory organization rather than arithmetic: one memory per (neuron slot, input) pair means that wider inputs or more parallel neurons multiply the block count, and packing several inputs per memory word is the natural extension.

8 Comparison with a Compiler-Generated Implementation

8.1 Method

The four networks of Table 4 were generated with hls4ml 1.3.0 [16] for the XC7Z020-1CLG484 at an 8 ns target, with Vitis HLS and Vivado 2025.2, and configured to reproduce the engine's arithmetic: `ap_fixed<16,8>` weights, biases and activations, `ap_fixed<40,24>` accumulation, truncation and saturation on the result, and weights pre-quantized by the loader's conversion. On 20,000 stimulus inputs the C emulation of each model agreed with the golden model on every raw output. Each network was generated with the `Resource` strategy at reuse factors RF $\in \{8, 16, 32, 64, 128\}$, and every design that fits the device was placed and routed out of context with an 8 ns clock constraint. The HLS timing estimate was not used: on the small networks points estimated at 5.75 ns routed at about 7.8 ns, and on the large ones at up to 11 ns.

8.2 Results

Table 12 and Fig. 7 report the result. Only four designs close timing at 125 MHz: three of hls4ml and the engine. On the two smaller networks hls4ml at RF 8 reaches 456 to 457 ns against 560 and 616 ns, 1.2 to 1.35 times faster, at a cost of 84 and 201 DSP slices for a single network. On the two larger networks no hls4ml point meets 125 MHz: the designs with enough parallelism exceed the LUT capacity or route at 91 to 125 MHz, and even at its own $f_{\max}$ the fastest hls4ml design takes 1026 ns against 768 ns for 40-64-32-16-3 and 1758 ns against 896 ns for 40-64-64-32-3.

Three qualifications apply, two in hls4ml's favor. The hls4ml designs were routed out of context, without the PS, DMA and interconnect that share the device with the engine. hls4ml pipelines successive inferences every II cycles by giving each layer its own multipliers, so its throughput is far higher; the engine cannot overlap inferences because the same 192 multipliers evaluate every layer in turn. For single-inference latency this does not help, and even the engine's 1.30 M inferences per second is two orders of magnitude above the 11,000 to 12,400 requests per second that the Linux host sustained with one request in flight. Finally, each hls4ml row is a separate bitstream with its weights compiled in, so changing or retraining a network costs a new synthesis and place-and-route run, from a few minutes to over half an hour per point here, whereas the engine runs all four networks from one bitstream at a fixed 192 DSP slices.

9 Discussion

The results support four observations. First, embedding the network in the PL makes the core latency a property of the pipeline: 768 ns with no spread, while the host path around it varies from 67 to 460 µs in median across operating systems. Second, $Q8.8$ is essentially accuracy-neutral here (at most 0.22 points across five models). Third, the design fits a low-cost Zynq-7000 at

Table 12 hls4ml implementations of the four networks on the XC7Z020 at an 8 ns target, against this engine. Latency is the HLS worst-case latency in cycles; f_{max} is obtained from the post-route worst negative slack, and * marks designs that close timing at 125 MHz; "Time" is latency at f_{max}. "Does not fit" marks designs whose synthesis exceeds the 53,200 LUTs of the device. The engine is a single bitstream that hosts all four networks; its cycle count is start to done and its LUT figure is for the engine alone.

Network	RF	Latency (cyc)	II (cyc)	DSP	LUT	f_{max} (MHz)	Time (ns)
40-10-16-8-3	8	59	10	84	5,624	129.0*	457
	16	87	20	39	7,045	118.5	734
	32	158	40	19	7,358	117.6	1,344
	64	270	80	10	8,044	97.6	2,766
	128	414	160	8	8,828	79.6	5,201
40-32-16-8-3	8	58	8	201	10,105	127.3*	456
	16	84	20	108	9,000	128.5*	654
	32	150	40	53	9,322	113.1	1,326
	64	254	80	27	13,941	101.9	2,493
	128	462	160	14	12,096	98.8	4,676
40-64-32-16-3	8	60	8	220	119,758	does not fit	
	16	93	20	220	31,940	90.6	1,026
	32	143	40	148	16,721	124.8	1,146
	64	278	80	73	23,943	105.7	2,630
	128	486	160	37	24,066	103.6	4,691
40-64-64-32-3	8	61	8	219	228,808	does not fit	
	16	94	20	220	91,808	does not fit	
	32	160	40	220	31,601	91.0	1,758
	64	262	80	131	28,935	103.5	2,531
	128	534	160	65	34,651	110.3	4,841
This work, all four	–	70 / 77 / 96 / 112	≥ lat.	192	13,038	125.9*	560 / 616 / 768 / 896

an estimated 2.70 W with positive slack. Fourth, run-time shape does not cost latency against a compiler that specializes the hardware: hls4ml is faster only on the two smaller networks at its most parallel points, while on the two larger networks the engine is 1.3 to 2.0 times faster than the fastest hls4ml design that fits. Specialization buys throughput, through inference-level pipelining that the engine does not implement.

The limitations are equally concrete. Only 14.4 µs of the 68.6 µs end-to-end latency is on the board, and the features are computed on the host, so the system as measured is not a complete trading path. The power figure is an estimate, the software baseline is scalar, and the evaluation uses one device and one task.

10 Conclusion

We presented an Ethernet-fed MLP inference engine in the programmable logic of a low-cost Zynq-7000. A 192-multiplier, 13-stage pipelined core performs a $Q8.8$ inference in 768 ns with zero timing variance, using 27.0 % of the LUTs and 87.3 % of the DSP slices of an XC7Z020 at an estimated 2.70 W, 0.97 W of it in the PL. The same network on the SoC's Cortex-A9 needs 52.4 µs, a factor of 68 on identical arithmetic, and the architecture accounts for a factor of 58.7 over an earlier sequential design at a common clock. The network shape is a run-time parameter: four topologies ran in one unchanged bitstream, each at its own cycle count, and against hls4ml on the same device, clock and arithmetic this costs no single-inference latency on the larger networks, where no hls4ml design closes timing at 125 MHz. Correctness was established bit-exactly over 400,000 inferences after directed simulation had exposed three pipeline-alignment defects.

Two directions follow. Integrating an Ethernet MAC in the PL would remove the host network stack from the dominant part of the path. Packing several inputs per memory word would relax the present bounds of four layers of 64 neurons and 64 inputs without additional DSP cost.

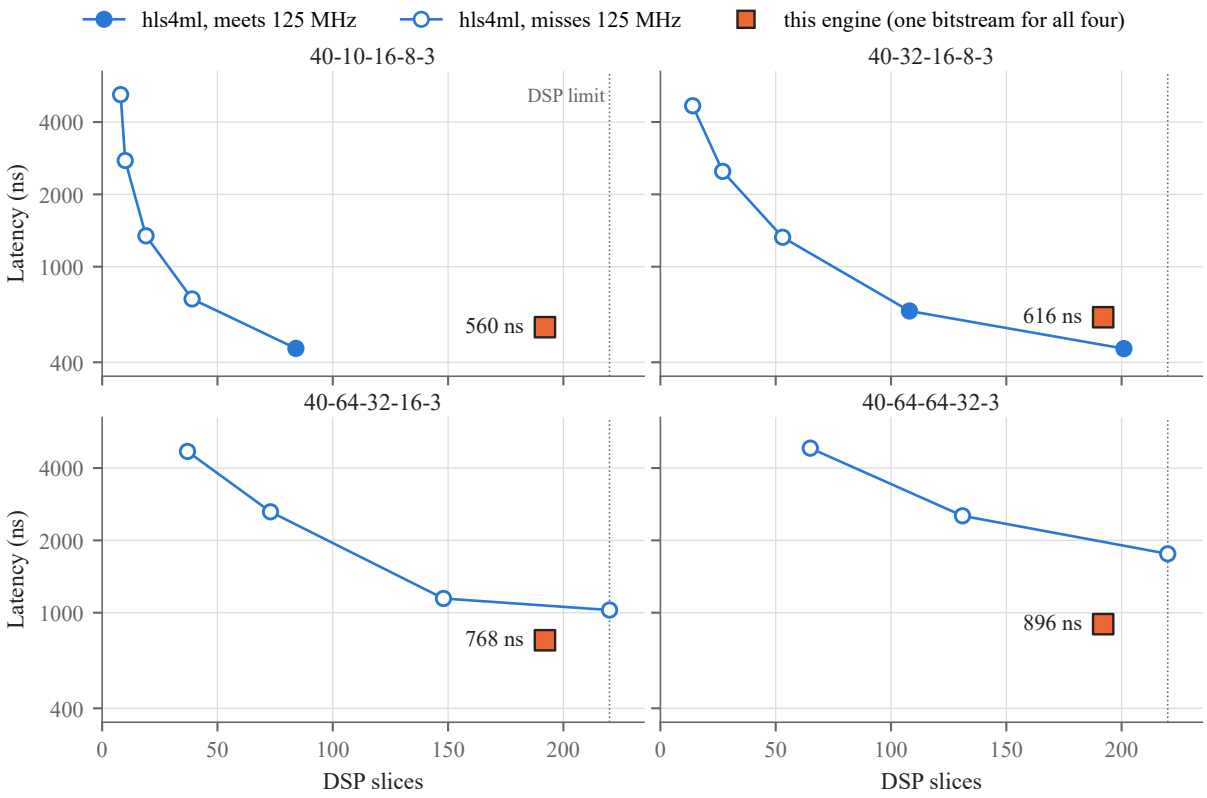


Fig. 7 hls4ml design points (RF 8 to 128) against the engine, one panel per network. Each hls4ml point is a separate bitstream; the engine (square) runs all four networks from one bitstream at 192 DSP slices. Filled hls4ml markers close timing at 125 MHz; latency is taken at each design's own f_{max}. Designs that exceed the device are omitted.

Statements and Declarations

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Competing interests. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Ethics approval and consent to participate. Not applicable.

Consent for publication. Not applicable.

Data availability. The market data used in this study are publicly available Binance BTCUSDT aggregated trade records. The measurement logs and verification stimuli are part of the artifact described under Code availability.

Code availability. The artifact supporting this study is publicly available at https://github.com/ozkanerbatuhan/hft (hardware) and https://github.com/ozkanerbatuhan/HFT_vitis (firmware and host tooling). It contains the RTL sources, the self-checking testbench, the cycle-accurate reference model, the host and firmware tooling, the stimulus generator, the post-implementation reports, and the measurement logs behind the latency and verification results. Instructions for rebuilding the bitstream and for reproducing each claim are given in the accompanying documentation.

Author contributions. Ahmet Batuhan Özkaner: conceptualization, methodology, hardware and software implementation, verification, formal analysis, investigation, data curation, visualization, and writing of the original draft. Mustafa Alper Ilık: software, investigation, and review and editing of the manuscript. Baykal Sarıoğlu: supervision and project administration. All authors read and approved the final manuscript.

Use of AI tools. Large language model based assistants were used as an aid in the literature search, in the

development of the measurement and verification software, and for correcting typographical and language errors in the manuscript. The authors reviewed all such output and take full responsibility for the content of this article.

References

[1] Budish, E., Cramton, P., Shim, J.: The high-frequency trading arms race: Frequent batch auctions as a market design response. The Quarterly Journal of Economics **130**(4), 1547–1621 (2015) https://doi.org/10.1093/qje/qjv027

[2] Aquilina, M., Budish, E., O'Neill, P.: Quantifying the high-frequency trading "arms race". The Quarterly Journal of Economics **137**(1), 493–564 (2022) https://doi.org/10.1093/qje/qjab032

[3] Leber, C., Geib, B., Litz, H.: High frequency trading acceleration using FPGAs. In: Proceedings of the 21st International Conference on Field Programmable Logic and Applications (FPL), pp. 317–322 (2011). https://doi.org/10.1109/FPL.2011.64

[4] Lockwood, J.W., Monga, M.: Implementing ultra low latency data center services with programmable logic. In: Proceedings of the IEEE 23rd Annual Symposium on High-Performance Interconnects (HOTI), pp. 68–77 (2015). https://doi.org/10.1109/HOTI.2015.20

[5] Puranik, S., Barve, M., Rodi, S., Patrikar, R.: Acceleration of trading system back end with FPGAs using high-level synthesis flow. Electronics **12**(3), 520 (2023) https://doi.org/10.3390/electronics12030520

[6] Brown, N., Klaisoongnoen, M., Brown, O.T.: Optimisation of an FPGA credit default swap engine by embracing dataflow techniques. In: IEEE International Conference on Cluster Computing (CLUSTER), FPGA for HPC Workshop (2021). https://doi.org/10.48550/arXiv.2108.03982

[7] Yoo, S., Kim, H., Kim, J., Park, S., Kim, J.-Y., Oh, J.: LightTrader: A standalone high-frequency trading system with deep learning inference accelerators and proactive scheduler. In: Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), pp. 1017–1030 (2023). https://doi.org/10.1109/HPCA56546.2023.10070930

[8] Zhang, Z., Zohren, S., Roberts, S.: DeepLOB: Deep convolutional neural networks for limit order books. IEEE Transactions on Signal Processing **67**(11), 3001–3012 (2019) https://doi.org/10.1109/TSP.2019.2907260

[9] Ntakaris, A., Magris, M., Kanniainen, J., Gabbouj, M., Iosifidis, A.: Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods. Journal of Forecasting **37**(8), 852–866 (2018) https://doi.org/10.1002/for.2543

[10] Zhang, C., Li, P., Sun, G., Guan, Y., Xiao, B., Cong, J.: Optimizing FPGA-based accelerator design for deep convolutional neural networks. In: Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA), pp. 161–170 (2015). https://doi.org/10.1145/2684746.2689060

[11] Qiu, J., Wang, J., Yao, S., Guo, K., Li, B., Zhou, E., Yu, J., Tang, T., Xu, N., Song, S., Wang, Y., Yang, H.: Going deeper with embedded FPGA platform for convolutional neural network. In: Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA), pp. 26–35 (2016). https://doi.org/10.1145/2847263.2847265

[12] Guo, K., Sui, L., Qiu, J., Yu, J., Wang, J., Yao, S., Han, S., Wang, Y., Yang, H.: Angel-Eye: A complete design flow for mapping CNN onto embedded FPGA. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems **37**(1), 35–47 (2018) https://doi.org/10.1109/TCAD.2017.2705069

[13] Umuroglu, Y., Fraser, N.J., Gambardella, G., Blott, M., Leong, P., Jahre, M., Vissers, K.: FINN: A framework for fast, scalable binarized neural network inference. In: Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA), pp. 65–74 (2017). https://doi.org/10.1145/3020078.3021744

[14] Blott, M., Preußer, T.B., Fraser, N.J., Gambardella, G., O'Brien, K., Umuroglu, Y., Leeser, M., Vissers, K.: FINN-R: An end-to-end deep-learning framework for fast exploration of quantized neural networks. ACM Transactions on Reconfigurable Technology and Systems **11**(3), 1–23 (2018) https://doi.org/10.1145/3242897

[15] Duarte, J., Han, S., Harris, P., Jindariani, S., Kreinar, E., Kreis, B., Ngadiuba, J., Pierini, M., Rivera, R., Tran, N., Wu, Z.: Fast inference of deep neural networks in FPGAs for particle physics. Journal of Instrumentation **13**(07), 07027 (2018) https://doi.org/10.1088/1748-0221/13/07/P07027

[16] Schulte, J.-F., Ramhorst, B., Sun, C., Mitrevski, J., Ghielmetti, N., Lupi, E., *et al.*: hls4ml: A flexible, open source platform for deep learning acceleration on reconfigurable hardware. ACM Transactions on Reconfigurable Technology and Systems **19**(2), 1–35 (2026) https://doi.org/10.1145/3801979

[17] Sun, C., Que, Z., Loncar, V., Luk, W., Spiropulu, M.: da4ml: Distributed arithmetic for real-time neural networks on FPGAs. ACM Transactions on Reconfigurable Technology and Systems **19**(1), 1–27 (2026) https://doi.org/10.1145/3777387

[18] Broglie, G., Morge-Rollet, L., Le Jeune, D., Le Roy, F., Roland, C., Canaff, C., Diguet, J.-P.: New LTE-A low-latency detectors for cognitive radio on low-cost FPGA SoC. Journal of Signal Processing Systems **96**(8-9), 479–493 (2024) https://doi.org/10.1007/s11265-024-01932-9

[19] Gdaim, S., Bensalem, H., Gagnon, G.: Efficient FPGA acceleration of deep neural networks for image processing using an OpenCL-based high-level synthesis framework. Journal of Signal Processing Systems **98**(2), 51 (2026) https://doi.org/10.1007/s11265-026-02014-8

[20] Rangsikunpum, A., Amiri, S., Ost, L.: Low-cost FPGA-based hybrid defence against sticker attacks for traffic-sign classification. Journal of Signal Processing Systems **98**(1), 20 (2026) https://doi.org/10.1007/s11265-026-01994-x

[21] Xia, M., Huang, Z., Tian, L., Wang, H., Chang, V., Zhu, Y., Feng, S.: SparkNoC: An energy-efficiency FPGA-based accelerator using optimized lightweight CNN for edge computing. Journal of Systems Architecture **115**, 101991 (2021) https://doi.org/10.1016/j.sysarc.2021.101991

[22] Tsoutsouras, V., Koliogeorgi, K., Xydis, S., Soudris, D.: An exploration framework for efficient high-level synthesis of support vector machines: Case study on ECG arrhythmia detection for Xilinx Zynq SoC. Journal of Signal Processing Systems **88**(2), 127–147 (2017) https://doi.org/10.1007/s11265-017-1230-1

[23] Wang, T., Wang, C., Zhou, X., Chen, H.: A survey of FPGA based deep learning accelerators: Challenges and opportunities. arXiv preprint arXiv:1901.04988 (2019) https://doi.org/10.48550/arXiv.1901.04988

[24] Rios-Navarro, A., Tapiador-Morales, R., Jimenez-Fernandez, A., Dominguez-Morales, M., Amaya, C., Linares-Barranco, A.: Performance evaluation over HW/SW co-design SoC memory transfers for a CNN accelerator. In: Proceedings of the IEEE International Conference on Electronics, Circuits and Systems (ICECS) (2018). https://doi.org/10.48550/arXiv.1806.01106

[25] Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., Kalenichenko, D.: Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2704–2713 (2018). https://doi.org/10.1109/CVPR.2018.00286

[26] Hubara, I., Courbariaux, M., Soudry, D., El-Yaniv, R., Bengio, Y.: Quantized neural networks: Training neural networks with low precision weights and activations. Journal of Machine Learning Research **18**(1), 6869–6898 (2017)

[27] Gholami, A., Kim, S., Dong, Z., Yao, Z., Mahoney, M.W., Keutzer, K.: A survey of quantization methods for efficient neural network inference. arXiv preprint arXiv:2103.13630 (2021) https://doi.org/10.48550/arXiv.2103.13630

[28] Al-Rikabi, H., Renczes, B.: Floating-point quantization analysis of multi-layer perceptron artificial neural networks. Journal of Signal Processing Systems **96**(4-5), 301–312 (2024) https://doi.org/10.1007/s11265-024-01911-0

[29] Moreau, T., Chen, T., Vega, L., Roesch, J., Yan, E.Q., Zheng, L., Fromm, J., Jiang, Z., Ceze, L., Guestrin, C., Krishnamurthy, A.: A hardware–software blueprint for flexible deep learning specialization. IEEE Micro **39**(5), 8–16 (2019) https://doi.org/10.1109/MM.2019.2928962

[30] Yu, Y., Wu, C., Zhao, T., Wang, K., He, L.: OPU: An FPGA-based overlay processor for convolutional neural networks. IEEE Transactions on Very Large Scale Integration (VLSI) Systems **28**(1), 35–47 (2020) https://doi.org/10.1109/TVLSI.2019.2939726

[31] Yu, Y., Zhao, T., Wang, K., He, L.: Light-OPU: An FPGA-based overlay processor for lightweight convolutional neural networks. In: Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA), pp. 122–132 (2020). https://doi.org/10.1145/3373087.3375311

[32] AMD: Vitis AI Development Environment. Product documentation (2024). https://www.amd.com/en/developer/resources/vitis-ai.html

[33] Khaki, A.M.Z., Choi, A.: Real-time and resource-efficient embedded computer vision via optimizing lightweight CNNs for FPGA acceleration. Journal of Signal Processing Systems **97**(2–4), 185–195 (2025) https://doi.org/10.1007/s11265-025-01966-7

[34] Kabir, E., Bakos, J.D., Andrews, D., Huang, M.: A runtime-adaptive transformer neural network accelerator on FPGAs. Microprocessors and Microsystems **120**, 105223 (2026) https://doi.org/10.1016/j.micpro.2025.105223

[35] Ratto, F., Máinez, A.P., Sau, C., Meloni, P., Deriu, G., Delucchi, S., Massa, M., Raffo, L., Palumbo, F.: An automated design flow for adaptive neural network hardware accelerators. Journal of Signal Processing Systems **95**(9), 1091–1113 (2023) https://doi.org/10.1007/s11265-023-01855-x

[36] Ezzaoui Rahali, H., Dave, A., Ruckman, L., Rahimifar, M.M., Therrien, A.C., Russel, J.J., Herbst, R.T.: Neural Network Acceleration on MPSoC Board: Integrating SLAC's SNL, Rogue Software and Auto-SNL. arXiv preprint arXiv:2508.21739 (2025). https://doi.org/10.48550/arXiv.2508.21739

[37] Boudjadar, J., Islam, S.U., Buyya, R.: Dynamic FPGA reconfiguration for scalable embedded artificial intelligence (AI): A co-design methodology for convolutional neural networks (CNN) acceleration. Future Generation Computer Systems **169**, 107777 (2025) https://doi.org/10.1016/j.future.2025.107777

[38] Umuroglu, Y., Akhauri, Y., Fraser, N.J., Blott, M.: LogicNets: Co-designed neural networks and circuits for extreme-throughput applications. In: Proceedings of the 30th International Conference on Field Programmable Logic and Applications (FPL), pp. 291–297 (2020). https://doi.org/10.1109/FPL50879.2020.00055

[39] Andronic, M., Constantinides, G.A.: PolyLUT: Learning piecewise polynomials for ultra-low latency FPGA LUT-based inference. In: Proceedings of the International Conference on Field Programmable Technology (ICFPT), pp. 60–68 (2023). https://doi.org/10.1109/ICFPT59805.2023.00012

[40] Andronic, M., Constantinides, G.A.: NeuraLUT: Hiding neural network density in boolean synthesizable functions. In: Proceedings of the 34th International Conference on Field-Programmable Logic and Applications (FPL), pp. 140–148 (2024). https://doi.org/10.1109/FPL64840.2024.00028

[41] Lou, B., Rademacher, R., Boland, D., Leong, P.H.W.: PolyLUT-Add: FPGA-based LUT inference with wide inputs. In: Proceedings of the 34th International Conference on Field-Programmable Logic and Applications (FPL), pp. 149–155 (2024). https://doi.org/10.1109/FPL64840.2024.00029

[42] Andronic, M., Constantinides, G.A.: NeuraLUT-Assemble: Hardware-aware assembling of sub-neural networks for efficient LUT inference. In: Proceedings of the IEEE 33rd

Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM), pp. 208–216 (2025). https://doi.org/10.1109/FCCM62733.2025.00077

[43] Khataei, A., Bazargan, K.: TreeLUT: An efficient alternative to deep neural networks for inference acceleration using gradient boosted decision trees. In: Proceedings of the ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA), pp. 14–24 (2025). https://doi.org/10.1145/3706628.3708877

[44] Gutermann, A., Serdyuk, A., Paraskevas, F., Toto Kiesa, H., Lesniak, F., Schwarz, J., Hartmann, M., Harbaum, T., Becker, J.: Joint exploration of neural networks and systolic hardware for improved AI accelerator performance. Journal of Signal Processing Systems **98**(2), 42 (2026) https://doi.org/10.1007/s11265-026-02012-w

Author Biographies

Photo

Ahmet Batuhan Özkaner received the B.Sc. degree in energy systems engineering in 2025 and the B.Sc. degree in electrical and electronics engineering (double major) in 2026, both from İstanbul Bilgi University, İstanbul, Türkiye. Research interests include FPGA-based digital design, low-latency hardware acceleration of neural network inference, and embedded systems.

Photo

Mustafa Alper Ilık is with the Department of Electrical and Electronics Engineering, İstanbul Bilgi University, İstanbul, Türkiye. Research interests include embedded systems and FPGA-based hardware design.



Baykal Sarıoğlu received the B.Sc. degree in electrical and electronics engineering from Kadir Has University, İstanbul, Türkiye, in 2004, and the M.Sc. and Ph.D. degrees in electrical and electronics engineering from Boğaziçi University, İstanbul, in 2007 and 2013, respectively. In 2007 Sarıoğlu worked as an FPGA system engineer at PAVO, and in 2013 was a postdoctoral researcher at Boğaziçi University within an EU FP7 project. Sarıoğlu joined the Department of Electrical and Electronics Engineering, İstanbul Bilgi University, in 2013 and has been an Associate Professor there since 2023. Research interests include analog and digital integrated circuits, biomedical circuits and systems, and embedded system design.